

Kalman Filter for Acoustic Localization: Experiments and Simulation

Final Project

E205: State Estimation for Robotics and Autonomy, Spring 2026.

Professor Adyasha Mohanty, *Harvey Mudd College*

Emily Barr
Harvey Mudd College
Claremont, CA
ebarr@hmc.edu

Sayema Lubis
Harvey Mudd College
Claremont, CA
slubis@hmc.edu

Abstract—Safe applications of robotics relies on the estimation of the robots state using integrated sensors, often excluding GPS in extreme environments. Without GPS, alternative sensors must be fused with the robots motion model to correct for accumulated position error over runtime. In this paper, a Kalman filter was implemented to estimate the position of an AlphaBot2-Pi robot using a microphone array to measure the time difference of arrival of specific frequency signals. A least-squares estimate and filtering allows microphone data to be fused with the robots constant velocity motion model, providing an estimate of the robots X and Y position within a 2D world frame. This problem was implemented in simulation and post-processed data collected on AlphaBot hardware using a minimum of three emitters and experimentally measured and tuned Q/R matrices for a variety of trajectories. Implementation of this type of sound localization within robotics can be applied to underwater and extraterrestrial environments, GPS denied areas, and alternative inverse applications that track sounds with multiple emitters.

I. BACKGROUND AND MOTIVATION

Sound localization techniques capitalize on sound movement as a waveform that travels linearly, so frequency emissions may hit a set of microphones at specific times based on array geometry. The time-difference between when the first emitted sound hits the array and other receivers provides insight into the directionality of the signal and its strength. Analysis of time-difference-of-arrival (TDOA) or time-of-flight (TOF) of acoustic signals are often employed in underwater navigation since low frequency acoustic signals travel well through water. This is a passive technique that collects accurate data via hydrophones that may be used to localize an emitter or receiver. This type of data collection is a viable sensor to implement within the measurement model of a Kalman filter to localize robots within a global environment. Fused with a known motion model, timeseries sound data can provide insight into the positioning of a robot within an emitter field based on the TDOA of different frequencies on a robot's onboard microphone array. This project investigates the viability of TDOA analysis and sound-based localization with the use of a linear Kalman filter on a wheeled AlphaBot. This

project was performed with on-land experimentation to mimic the application of acoustic processing and mapping in water or in low-noise environments. Using acoustic processing methods such as short-time Fourier transforms, low-pass filtering, and amplitude analysis, this project fuses methods used in State Estimation with digital signals processing to investigate sound-based localization as a viable method for robotic navigation in on-land applications.

II. METHODS

A. Motion Model

The robotic platform used within this project is an AlphaBot2, a wheeled robot capable of dynamic velocity and acceleration. This robot will track its state as an X, Y coordinate within a 2D world. Assume that the world frame N is fixed and the robot moves within positive and negative coordinates in reference to its fixed origin.

For simplicity, the motion of this AlphaBot is constrained to a constant velocity model. These dynamics may inform the state via a measurement model described in Section II, B. The matrix F maps the motion model to the state, allowing the state (X, Y) to be directly influenced by the robot's motion.

Constant Velocity Equations: $p_{t+1} = p_t + v_t \Delta t$

where $v_x = v_{pwm} \cos(\theta)$ $v_y = v_{pwm} \sin(\theta)$

due to the rotation of θ of the robot's body frame within the world frame N .

$$\begin{bmatrix} x_{t+1} \\ y_{t+1} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \cdot \begin{bmatrix} x_t \\ y_t \end{bmatrix} + \begin{bmatrix} \Delta t & 0 \\ 0 & \Delta t \end{bmatrix} \cdot \begin{bmatrix} v_x \\ v_y \end{bmatrix}$$

The constant velocity model maps the current position P to the state X and Y by adding a $v_t \Delta t$ to the current position. This is seen in the direct mapping from measurements to x_t and y_t from the motion model, represented with the identity matrix, and an additive term of the time change multiplied with constant v .

B. Measurement Model

Since the robot will use audio data as a way to estimate its position, the measurement model consists of a pipeline that converts raw audio data into a 2D position estimate. Time Difference On Arrival (TDOA) utilizes the difference in arrival times of an acoustic signal across multiple microphones. These time differences depend on the relative geometry between the sound source and the microphone array. Each TDOA measurement defines a hyperbola representing the set of robot positions that would produce the observed time delay (seen in Figure 1) [7]. With multiple microphones, several such hyperbolic constraints can be formed and the robot's position corresponds to the point that best satisfies all of these constraints.

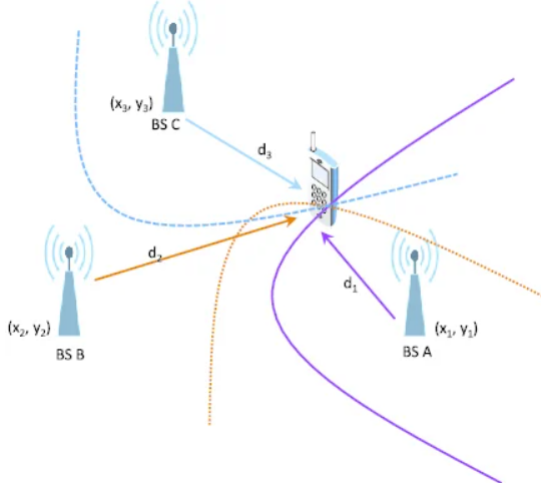


Fig. 1. Illustration of TDOA-based localization. A robot equipped with a microphone array receives acoustic signals from multiple beacons at known positions (x_1, y_1) , (x_2, y_2) , (x_3, y_3) . Differences in signal arrival times across microphones define hyperbolic constraints (shown as curves), and the robot's position is estimated as the point that best satisfies these constraints. Photo taken from [2]

In practice, due to noise in the measurements, these hyperbolas do not intersect at a single point. Instead, the localization problem is formulated as a nonlinear least-squares optimization problem. For a candidate robot position (x, y) , the expected TDOA values are computed based on the known microphone array geometry and beacon position. These predicted TDOA values are then compared to the measured values obtained from cross-correlation of the microphone signals.

The measurement residual is defined as:

$$e(x, y) = z_{\text{meas}} - h(x, y)$$

where z_{meas} represents the measured TDOA vector and $h(x, y)$ is the predicted TDOA given a candidate position. The estimated robot position is obtained by minimizing the squared error:

$$\min_{x,y} \|z_{\text{meas}} - h(x, y)\|^2$$

Ultimately, the measurement out of TDOA and least-square solver pipeline is a x and y position therefore the final H matrix is,

$$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

C. Hardware Modifications

To implement experimentation aboard the AlphaBot, the robotic platform needed to be outfitted with proper sensors for TDOA measurements and data collection. The AlphaBot was outfitted with INMP44 microphones capable of streaming data captured over I2S. The microphone array consisted of four evenly spaced INMP44s, paired to stream data into a Teensy. One teensy consolidated data from each pair of microphones, then transmitted the fused data stream to the Raspberry Pi via serial UART USB connection.

To attach the microphones to the AlphaBot, acrylic mounts were machined in the HMC machine shop (Figure 2, 3). The design of these microphone mounts allowed for soldered connections onto the attachment pads on the PCBs while keeping the microphone oriented upwards and concentrically aligned with the screw holes/standoffs between the two PCBs on the AlphaBot (Figure 3). A through hole allows the microphone mounts to thread onto the male part of the standoffs while a screw attaches from the bottom of the PCB into the standoffs female side (Figure 4). Utilizing the preexisting standoff threads as an attachment point anchors the microphones to known locations on the AlphaBot, as these locations are easily measurable with calipers and are present on open-source CAD models of the robotic system [5]. The anchor points are symmetric across the X and Y axis passing through the robot's origin, the center point on the robot body (Figure 5). The standoff anchor points are 3.3 inches apart in the X direction and 2.8 inches apart in the Y direction, both directional dimensions are spaced equidistant from the robots origin (ie: each mount is 1.15 inches away from the origin in X (Figure 5) [5].

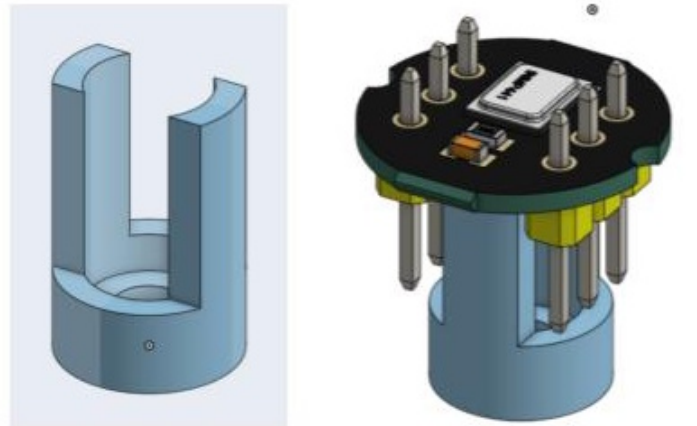


Fig. 2. Microphone mount CAD. The mount was designed to support the microphone from the bottom, allowing the microphone to remain unobstructed during testing. INMP44 Microphone CAD from [6]



Fig. 3. Manufactured acrylic mount from scrap. This design was manufactured using lathe and milling processes and successfully screwed onto the standoffs in the AlphaBot.

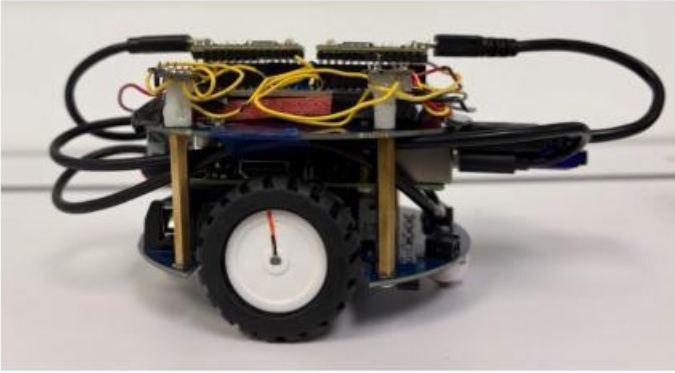


Fig. 4. Upward orientation of microphone on top of the threaded standoff attachment. Standoff is mounted female-side-down and attaches to the bottom PCB on the AlphaBot with a phillips head screw.

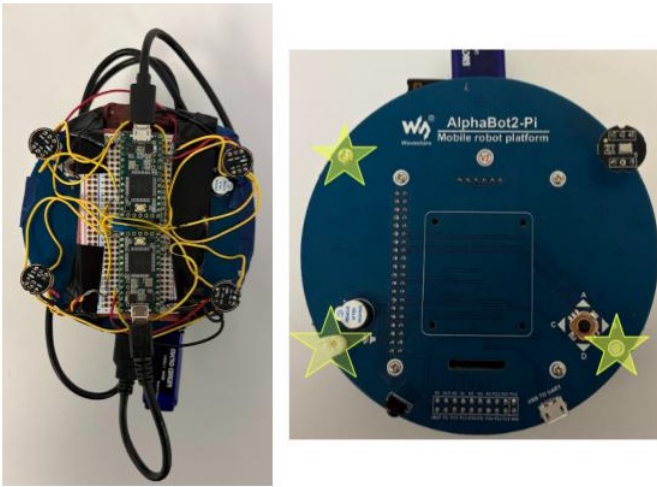


Fig. 5. Layout of the microphone mounts aboard the AlphaBot. These anchor points above the standoffs ensure consistent, symmetric spacing between the mounts in the X and Y directions.

Serial communication between the Teensy microcontrollers and the Raspberry Pi was used to stream audio data from the microphone array. The Teensys were first programmed using the Arduino IDE, where firmware was uploaded to initialize the microphones and enable continuous audio data transmission. Once deployed, each Teensy began sending sampled audio data over USB serial to the Raspberry Pi.

On the Raspberry Pi side, a Python-based interface was implemented to establish and manage the serial connections. The serial ports were opened once during the program initialization, after which the interface continuously monitored the incoming data streams. At a fixed interval of 0.1 seconds, the program reads the most recent line available from each serial buffer. This allows for a lightweight approach to sampling the ongoing data stream, reducing potential errors during data collection. The collected data was then written to CSV files for offline analysis and processing. To coordinate data collection with robot behavior, timestamps were generated using Python's `time.monotonic()` function, which provides a steadily increasing clock unaffected by system time adjustments. These timestamps were used to identify when the robot stopped moving and entered a listening phase, allowing recorded audio segments to be aligned with specific waypoints during post-processing.

D. Simulation Environment

The simulation was developed to process and evaluate experimental data using the Kalman filter. The goal of this simulation was to emulate a real-time localization pipeline, allowing for controlled analysis of how the system behaves under varying measurement conditions. In particular, the simulation enables comparison between the robot's true trajectory, the noisy TDOA-based position estimates, and the filtered state produced by the Kalman filter.

The simulation was written in Python and implemented within Google Colab for ease of collaboration, sharing, and reproducibility. It utilizes standard libraries such as NumPy for numerical operations, SciPy for optimization (the least-square estimation), and Matplotlib for visualization.

The simulation framework integrates several key components developed in this project. First, the TDOA measurements are generated synthetically by adding random noise to the true time delays, controlled by a tunable "TDOA error" parameter. Then, these time-delay measurements are passed into a nonlinear least-squares solver, which estimates the robot's position based on the known beacon and microphone geometry.

The resulting position estimates are then used as measurements in a Kalman filter, which combines the motion model and measurement model to produce a refined estimate of the robot's trajectory. The Kalman filter is implemented in a standard predict and update framework, where the prediction step propagates the state using the motion model, and the update step corrects this prediction using the least-squares position estimates.

The overall structure of the simulation pipeline is summarized in the pseudocode below:

```

for each timestep k:

    # Prediction step
    $\\$
    x_pred = A x_prev + B u
    P_pred = A P_prev A^T + Q

    # Measurement step
    # (TDOA + least squares)

    z_tdoa = compute_tdoa(audio_window)
    z_meas = least_squares_position(z_tdoa)

    # Update step
    y = z_meas - H x_pred
    S = H P_pred H^T + R
    K = P_pred H^T S^{-1}

    x_est = x_pred + K y
    P_est = (I - K H) P_pred

```

This simulation framework enables systemic evaluation of the localization pipeline and provides insight into how measurement noise, model assumptions, and system design choices affect overall performance.

E. Calibration and Bias Removal

1) *Speed calibration*: As the AlphaBot's speed operates in an unknown PWM unit, it was important to perform a speed calibration to relate the PWM input to the robot's velocity. Due to the additional onboard sensors and boards that provide additional weight to the robot, simple estimates based on wheel rotations over time were found to be unreliable. Therefore, an experimental calibration approach was done instead.

The robot was programmed to move forward for 5 seconds 5 times at three different PWM values (10, 20, and 30) and its displacement was recorded where the velocity can be computed through d/t where d is the displacement and t is the time taken to finish distance d (see Figure 6). As shown in Figure 7, the measured velocities increase approximately linearly with PWM input. A linear fit was applied to the averaged velocity data, resulting in the following relationship:

$$v = 0.541 \cdot \text{PWM} - 0.161$$

with a $R = 0.998$, indicating a strong linear relationship. This calibrated model was used to define the control input in the motion model for the Kalman filter.

2) *Finding Experimental Q and R* : The motion covariance Q and measurement noise covariance R are tunable parameters in a standard Kalman filter, however, tuning them without empirical data can lead to a poor representation of the real system. These parameters should reflect the true uncertainties in the system, which are often difficult to model analytically. In this work, both Q and R are empirically derived from experimental data to capture real-world effects, allowing the

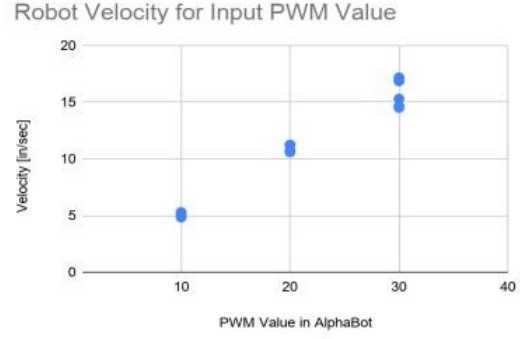


Fig. 6. Measured robot velocities at different PWM input values. Each point represents the velocity computed from displacement over time of a given PWM command, illustrating the trend of increasing speed with higher input.

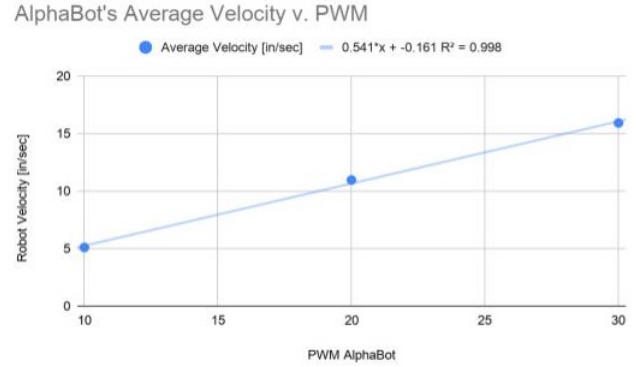


Fig. 7. Average robot velocity as a function of PWM input with a linear regression fit. This fit shows a strong linear relationship ($R^2 = 0.998$) and is used to parameterize the motion model.

filter to more accurately balance trust between the motion model and measurements.

Q – Motion Covariance

To estimate the motion model covariance Q , the robot was commanded to complete a straight-line trajectory (along $y = 0$) with a PWM speed of 10 and its final positions were recorded over 23 trials, as shown in Figure 8. Ideally, the robot would remain on the line, however, the recorded positions show noticeable spread and a consistent drift away from the desired end position.

These deviations reflect real-world effects and the robot's hardware limitations such as friction, motor imbalance, and surface interactions. The distribution of the recorded positions in the x and y directions was used to compute a process noise covariance Q in the Kalman filter. This empirically derived Q captures the variability in the robot's motion and allows the model to better reflect its true behavior.

The final Q matrix is,

$$Q = \begin{bmatrix} 3.95 \times 10^{-3} & 0 \\ 0 & 2.99 \times 10^{-3} \end{bmatrix} \quad (1)$$

R – Measurement Covariance

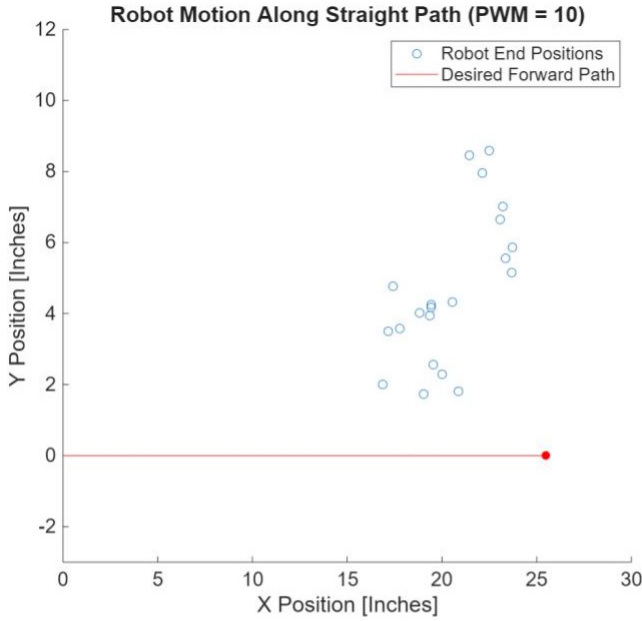


Fig. 8. Calibration of the motion model covariance Q . The robot was programmed to execute a straight-line trajectory along $y = 0$ (red line), while the recorded end positions (blue markers) show significant deviation from the desired path. This spread in the observed positions reflects systematic motion errors, which were used to estimate the motion covariance Q .

The measurement covariance matrix R was estimated experimentally to characterize the noise in the TDOA-based localization pipeline. Since R represents uncertainty in the position measurements, an experiment was designed to isolate measurement noise by removing motion-related variability.

In this experiment, the robot was held stationary while a single acoustic beacon emitting a constant 200Hz tone was placed at known distances of 1ft, 2ft, and 3ft recorded, and the experiment was repeated three times, resulting in a total of nine audio recordings. Because the ground truth beacon positions were known and the robot remained static, any variation in the estimated robot position can be attributed to measurement noise in the sensing pipeline.

To increase the number of measurement samples, each audio recording was segmented into overlapping time windows. A window duration of 1 second was used, with a 0.5-second overlap between consecutive windows. For each window, TDOA values were computed using cross-correlation between microphone signals. Specifically, three relative time delays were calculated with respect to a reference microphone: τ_{12} , τ_{13} , and τ_{14} .

Each set of TDOA measurements was then passed through the nonlinear least-squares solver to estimate the robot's position relative to the known beacon location. This produced a collection of position estimates for each window, all corresponding to the same true robot position.

The measurement covariance matrix R was then computed as the sample covariance of these estimated positions:

$$R = \text{cov}(Z_{\text{pos}})$$

where Z_{pos} is the set of all position estimates obtained across all windows, trials, and distances. This covariance captures the variability introduced by noise in the audio acquisition, synchronization errors, and TDOA estimation. As such, it provides an empirical estimate of the uncertainty associated with the measurement model used in the Kalman filter.

The empirical R matrix is

$$\begin{bmatrix} 4.63 \cdot 10^{10} & 7.40 \cdot 10^6 \\ 7.40 \cdot 10^6 & 2.39 \cdot 10^5 \end{bmatrix}$$

The resulting measurement covariance matrix has a large magnitude, indicating significant variability in the position estimates derived from the TDOA pipeline. While the entries of R are relatively large, the matrix remains a valid covariance matrix, as it is symmetric and positive. The large values observed in R reflect substantial measurement noise in the system, arising from factors such as microphone synchronization error, serial communication latency, and inconsistencies in the audio data. These sources of error introduces significant variability into the TDOA measurements, which propagates through the least-squares solver into the final position measurements.

A more detailed discussion of these error sources and their impact on system performance is provided in Section V.

III. EXPERIMENTAL SET-UP

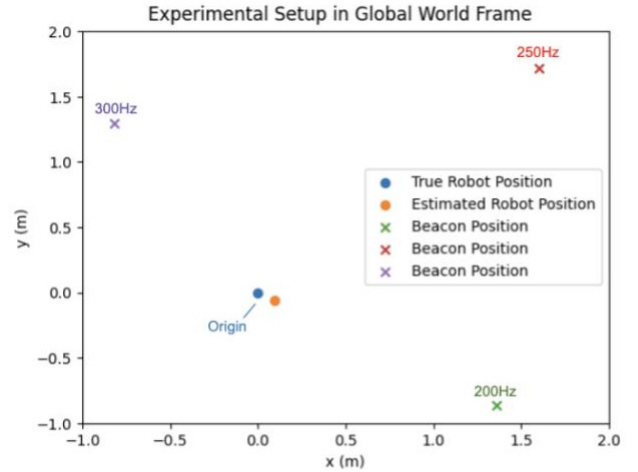


Fig. 9. Placement of beacons relative to the origin and initialized robot position in simulation and physical trials.

In each experiment, the robot started at the origin of the world frame and traveled in a straight line towards the 250Hz beacon. The robot stops and listens for the beacons at each waypoint along the robot's motion. This is seen as randomly generated TDOA measurements in the simulation and physically stopping driving and collecting audio data over 20 seconds during physical trials. In both of these experiments (simulation and physical), the robot and beacon placement remained static at initialization ($t = 0$) (Figure 9).

In physical experiments, two different trajectories were tested over 1-3 trials (Figure 10). One experiment required

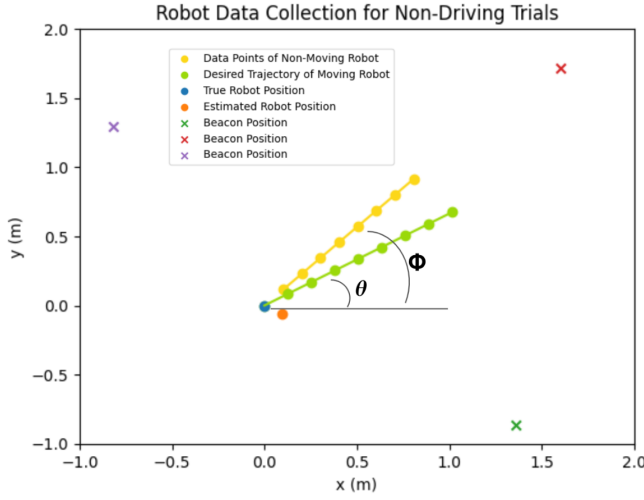


Fig. 10. Experimental set-up of the robot localization where the green points indicate the “moving” robot trajectory (Experiment 1) and yellow points indicate “static” but manually moved robot trajectory (Experiment 2). The constant values for $\theta = 33.6$ and $\Phi = 48.5$. This illustrates the idealized trajectories of the experiments.

that the robot drive roughly 4 feet, stopping every 6 inches to listen and collect beacon data for 10 seconds. This relied on the motion of the robot to move through the given trajectory.

The second experiment included non-driving trials, to abstract the reliance on the robots driving methods during data collection. Instead of driving, stopping, and collecting data at waypoints, the robot was manually moved through a series of 9 waypoints, every 7 inches to collect data instead. Therefore, the robot’s true trajectory (ground truth) was known, since it was manually moved to a known position at each waypoint. Manually moving the robot also mitigated errors from the AlphaBots wheel motion during driving trials. Often, the robot’s left wheel would fail in the middle of its program, leading the command `Ab.forward()` to cause the robot to rotate on the spot. This second experiment type was done to idealize this linear trajectory despite issues with the AlphaBots driving.

The first experiment occurred at an angle of 48.5 degrees, whereas the second experiment occurred at 33.6 degrees. These trials, relative to the origin and beacon positions, are modeled in Figure 10 with accurate waypoint spacing.

A summary of experiments is below:

- 1) First experiment – Driving Robot Trajectory
 - a) PWM = 10
 - b) Waypoints of 6, 12, 18, 24, 30, 36, 42, 48 inches (8 datapoints)
 - c) Drives for 5 seconds, stops and listens for 20 seconds for beacons to fire in series (200, 250, 300Hz)
- 2) Second experiment
 - a) PWM = 0, no wheel motion
 - b) Waypoints of 7, 14, 21, 28, 35, 42, 49, 56, 63 inches (9 datapoints)

- c) Robot manually moved to each waypoint, listens for 20 seconds for beacons to fire in series (200, 250, 300Hz)

The first and second physical experiments were run between Sprague and Parsons at Harvey Mudd College between 9-11AM on May 4th, 2026 (Figure 11).



Fig. 11. Location of data collection for outdoor physical experiments (experiments 1 and 2).

IV. EXPERIMENTAL RESULTS

A. Simulation Results

Q and R were then plugged into the digital Kalman filter implementation to simulate the robots motion with realistic measurement and motion covariance. These Q and R matrices were featured in the simulation to both verify that the filter was working and assess the differences between tuning and experimentally found covariance.

Since the R that was found experimentally was quite large, it was tested in simulation both as the experimental value and a tuned value. Q was incorporated into the simulation as the squared X and Y covariance of the robot motion within a diagonal matrix that remained constant through all simulations.

Realistic R Simulation Plots

The experimentally found R matrix was plugged into the simulation to model the robots path, actual trajectory, and TDOA measurements. This matrix, with values on the order of 10^{10} , reflects extremely high uncertainty in the position estimates obtained from the TDOA pipeline. Two simulations were conducted: one with high TDOA noise (on the order of milliseconds) and one with very low TDOA noise (on the order of picoseconds), while keeping the same experimentally derived R .

As shown in Figures 12 and 13, the Kalman filter produces nearly identical predicted trajectories in both cases, despite the large difference in TDOA noise levels. In both scenarios, the estimated trajectory remains approximately linear, not capturing the noisy true robot path. This behavior indicates that the filter places significantly more weight on the motion model

than the measurement updates. This motion model trajectory dominates the final Kalman filter estimate.

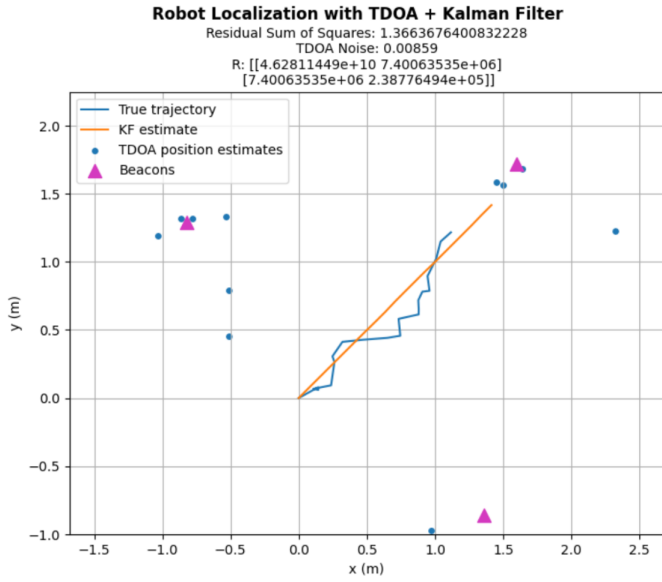


Fig. 12. TDOA noise is high, 8.59ms. Estimates from the Kalman filter should be bad, but experimental R value dominates the simulation so the Kalman filter relies on a motion model. This output strays from the true trajectory of the robot.

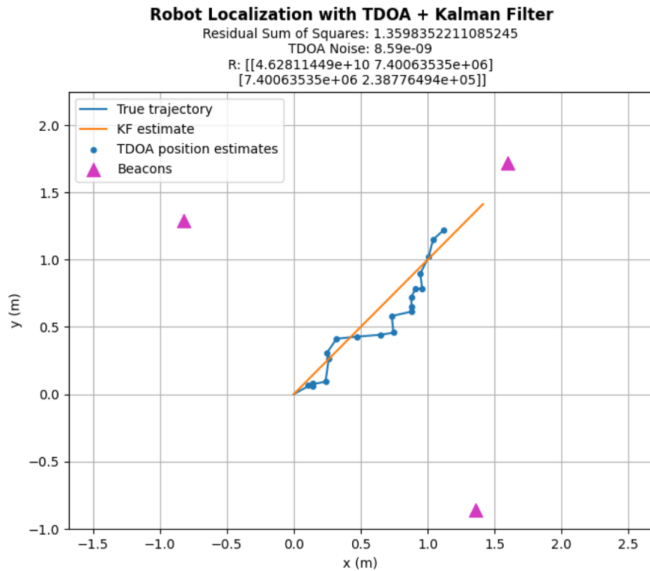


Fig. 13. TDOA noise is low, 8.59ps. Estimates from Kalman filter should be quite good, considering accurate measurements. Experimental R still dominates the simulation so the Kalman filter relies on a motion model more than the accurate measurements. This output strays from the true trajectory of the robot and matches the results from a high TDOA noise in Figure X.

These results highlight a key limitation of the system: the experimentally derived measurement covariance is so large that it renders the measurement model ineffective within the Kalman filter framework. This outcome directly reflects the high variability observed in the TDOA-based position mea-

surements and emphasizes the importance of improving sensor reliability and synchronization to obtain a more informative measurement covariance.

Simulated R Formation (TDOA Affects)

Since a high experimental R value meant that the motion model was dominating the simulated Kalman filter approximation of the robots path, R was then tuned to see the effect on TDOA noise on the state estimate using the simulation. This experimental approach ideally would show that low TDOA noise would improve the TDOA estimates, which are being relied on heavily when R is tuned lower than the experimentally found R .

To run these simulations, R was chosen to be $1e-6$ seconds, one microsecond, as the covariance in TDOA measurements. TDOA was then varied between values of $8.59e-5$, $2e-5$, 0.000859 , and $8.59e-9$ seconds. $8.59e-4$ seconds is roughly 3.5 times larger than the time it takes for sound to travel 2.8 or 3.3 inches (0.083 - 0.071 meters) from one microphone to the other in the X or Y direction. This value was arbitrarily chosen to be varied in powers of 10 to show the effects of TDOA error.

See that with a tuned R , the simulation “trusts” the measurements more, factoring them into the Kalman filter estimate of the robot’s state (X, Y). The lower the TDOA error, the closer the measurements match the true path, allowing the Kalman filter to properly capture the motion of the robot.

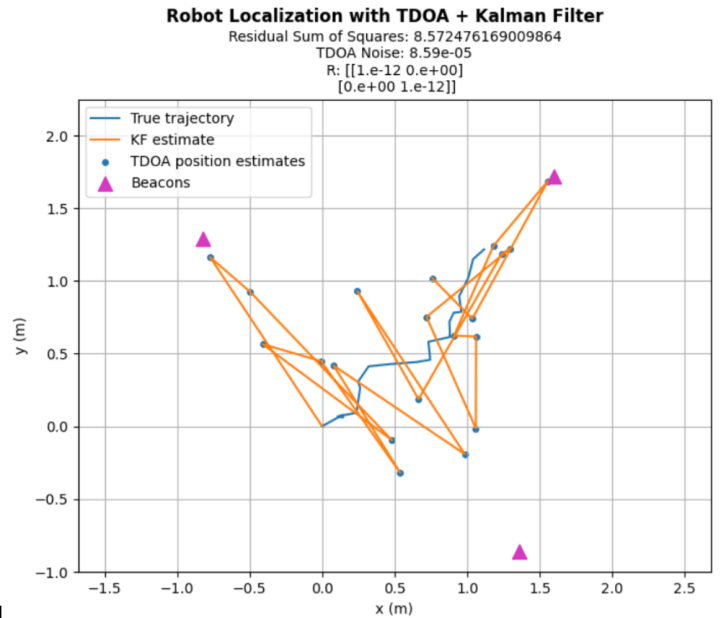


Fig. 14. TDOA noise is $8.59e-5$ seconds. This is not a fine enough resolution to correctly situate the measurements on the robots path. Since the Kalman filter thinks that these are reliable measurements (tuned R), the Kalman filter incorrectly maps the trajectory of the robot (orange).

Figures 14-16 illustrate the effects of variable TDOA noise on the simulated results. By decreasing TDOA noise (which would come from expensive, accurate hardware), position estimates improve when the model is able to rely on TDOA

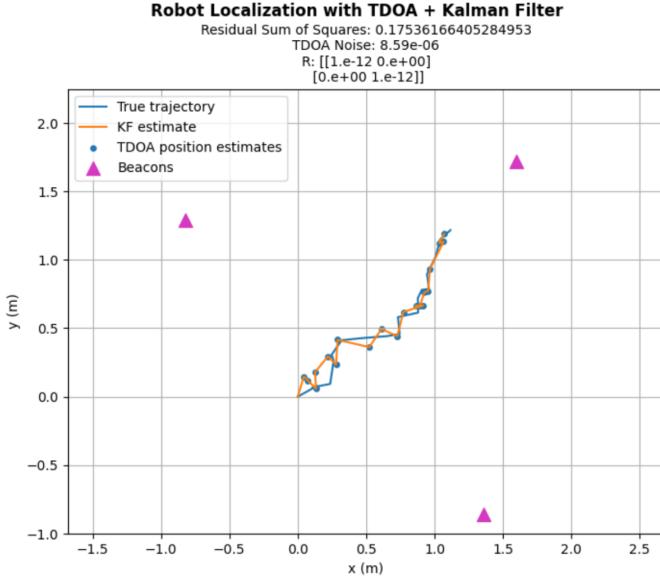


Fig. 15. TDOA noise is 8.59 microseconds. This is fine resolution data collection that decreases the spread of data points around the robots general path, as seen with the $2e-5$ second noise in figure X. This noise value allows the Kalman filter to converge faster to the true path since the data points are already close to the robot's trajectory.

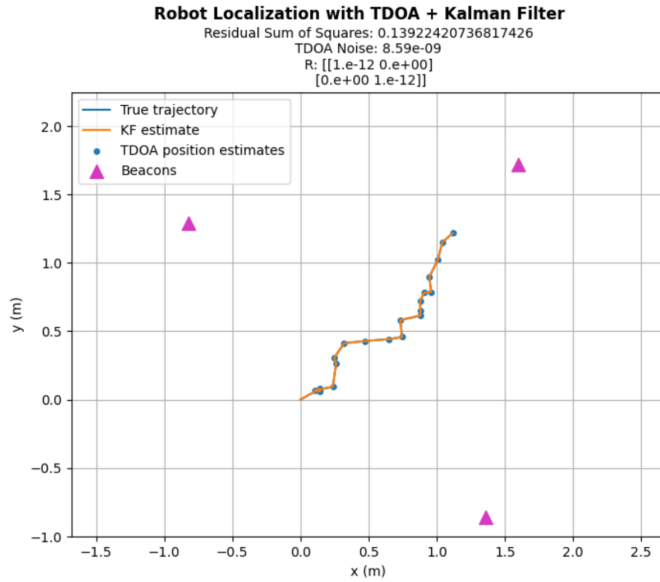


Fig. 16. TDOA noise is 8.59 picoseconds. This is extremely fine resolution data collection that centers the data points collected exactly on the true path of the simulated robot. This error being extremely low means that the Kalman filter is accurate immediately, as soon as the robot begins motion from the origin. The filter doesn't converge after a certain number of datapoints since the hardware is extremely accurate, leading to an extremely accurate position estimate.

data. Therefore, this simulation validates that with more reliable hardware (that would decrease TDOA noise and R value), the simulation would be able to properly factor in both measurements and the motion model to the state estimate, unlike what was seen in the section on realistic R simulation plots from the experimental R .

B. Post-Processing Results

After conducting the experiments described in Section III, the data collected were processed in the TDOA and least squares solver pipeline to generate the position estimates. They were then used as inputs to the Kalman filter that uses all the following parameters described in Sections II,E and III.

Post-Processing Method

As mentioned in Section II, C, the set-up only allows for sampling every 0.1 seconds, in other words, the system has a sampling rate of 10 samples per second. This low sampling rate stems from issues in the method of data collection from the INMP44 microphones streaming via serial into the RaspberryPi onboard the AlphaBot. INMP44 microphones had numerous issues streaming reliable data into the AlphaBot, therefore requiring a low sampling rate as to not overload the program streaming serial from two separate Teensys, one per pair of INMP44s. However, the sample rate of the serial capture program should have been much higher, according to Nyquist theorem, which drives the sampling rate to be two times more than the highest frequency of data that is being collected. Therefore, with an emitter of 300Hz, the program should have been sampling as fast as 600Hz in order to properly capture the onset of the 300Hz beacon on a proper timeseries. Since the data collection rate was too low, the STFT performed on captured data was not fine enough resolution to show the arrival of each beacon blast at each microphone (Figure 17).

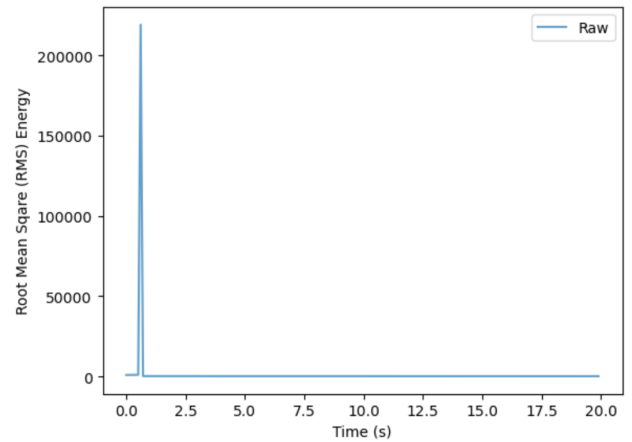


Fig. 17. One data point from Mic 2 that shows a single peak instead of a continuous stream of audio data with three peaks indicating the three beacons firing.

Despite the faulty datapoints, the post-processing pipeline was still developed in order to evaluate the Kalman filter

under realistic sensing conditions. Due to the sampling rate being too low, Fourier analysis was unavailable hence, seeing volume changes was the method to distinguish the different segments of when the three beacons fire. To get the volume changes, the root mean square (RMS) of the audio data is taken then made negative to get the peaks of the minima. The audio signal was first smoothed to reduce noise and avoid mischaracterizing local fluctuations as meaningful minima. This step helps ensure that the detected minima more accurately correspond to the transitions between beacon segments. A Gaussian filter with a standard deviation of $\sigma = 2$ was applied, which preserves the overall shape of the signal while reducing high-frequency noise. Additional constraints were introduced to further improve the segmentation reliability. Minima were not allowed to be detected within the first and last second of the audio recording. Furthermore, a minimum separation of 4 seconds between detected minima was enforced to ensure that each identified segment corresponds to a distinct beacon emission. An example of this segmentation process can be seen in Figure 18.

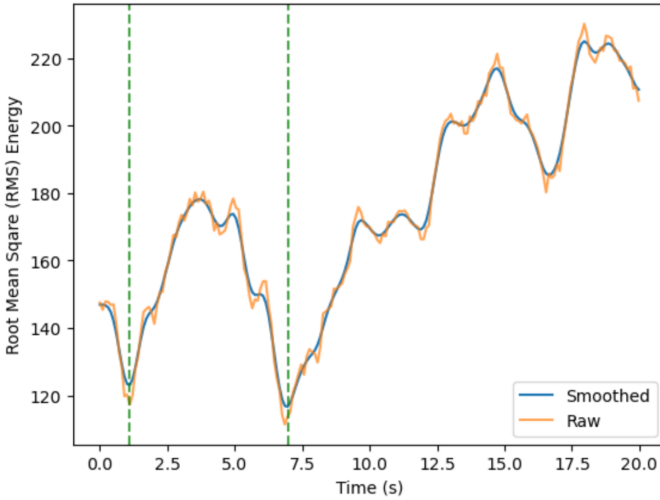


Fig. 18. Volume of an audio recording where the raw data is shown in orange and the smoothed version is shown in blue. The green dashed line represents the cuts of where the segments are made. There will always be three segments in total as it corresponds to a beacon emission.

Kalman Filter Results

The following measurements are then fed to a Kalman filter, which is parameterized according to the experimental set-up described in Section III.

As shown in Figure 19, the raw position estimates obtained from TDOA are highly inconsistent and deviate significantly from the expected robot trajectory. Many of these measurements are physically implausible, appearing far from both the robot's path, and the known beacon locations, indicating substantial noise and error in the data acquisition stage (Figure 19).

Despite the poor quality of these measurements, the Kalman filter produces a smooth, approximately linear trajectory that closely follows the predicted motion model. This is a direct

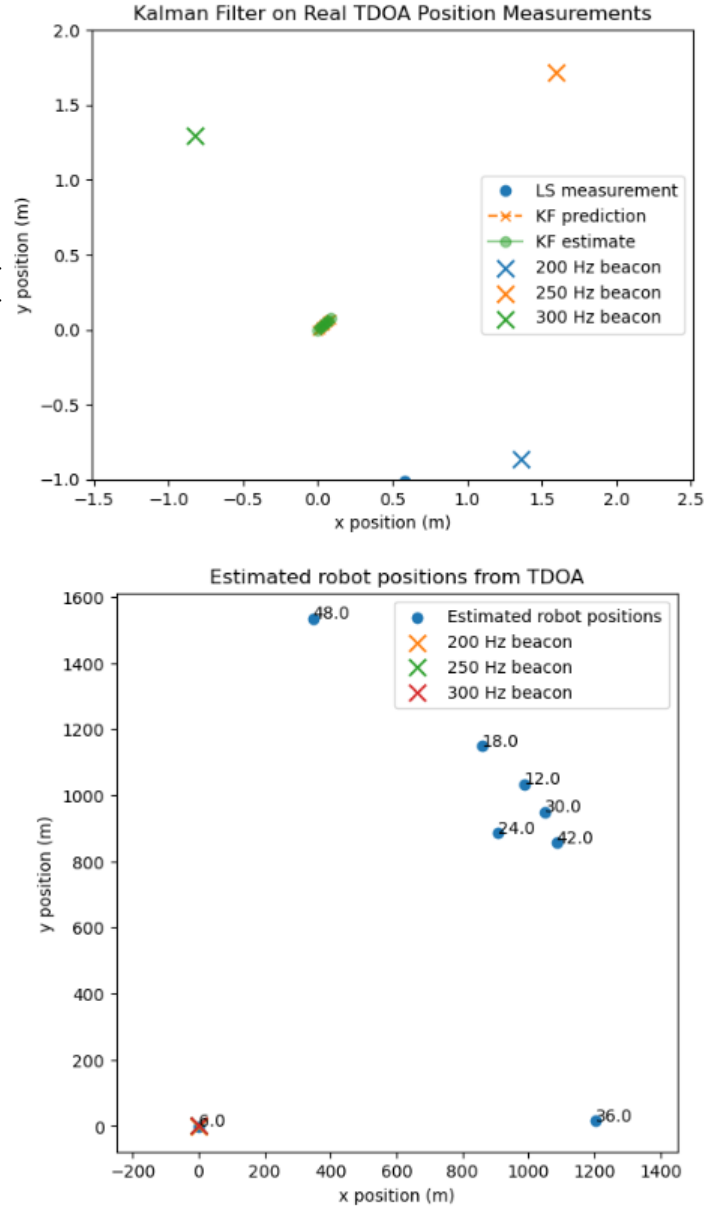


Fig. 19. a) Post-processed results on the measurements taken experimentally which shows the Kalman position estimate before the prediction step (green dots connected by line) and the predicted state (orange x connected by line). Three Xs of varying colors show positions of the beacon. b) A zoomed out view of the same figure to show where the least-squares measurements are. As the measurements are significantly far from the predicted trajectory, this shows how unreliable the audio data was during the sensing process.

consequence of a large experimentally derived measurement covariance R . Because R is several orders of magnitude larger than the motion model noise Q , the Kalman gain becomes very small, causing the filter to heavily down-weight measurement updates. As a result, the filter relies almost entirely on the model, effectively ignoring the noisy TDOA-based position measurements.

V. DISCUSSION AND CONCLUSIONS

As seen in the previous Section IV, the current hardware for audio acquisition is a huge source of error that resulted in unreliable measurements and overdependence on the motion model in the Kalman filter. The sampling issues, unreliable microphones, and unstable serial connections caused failure in the initial use of the STFT and low-pass filtering of the project. Filtering out background noise via low-pass filtration and running an STFT to see frequency over time is the correct, more reliable approach for processing the data collected in this project.

Failure of these methods due to coarse data collection caused the pivot to using windowing and amplitude analysis, which is a less-optimal method of assessing the time of arrival per signal. Instead of picking out which beacon arrived at specific times, amplitude analysis relies on the increase of volume when a beacon is fired. This approach was only viable in this application since there is silence between each beacon and they fired in a consistent order within the 10 seconds of collection time. If this experimental setup was more random, this amplitude analysis wouldn't have worked to properly deduce time of arrival since there is no indicator of which frequency arrives at a particular time, since there isn't a reliable frequency analysis.

Additionally, the time of arrival data is extremely coarse, since the 10Hz sampling rate makes the data only occur in increments of 0.1 seconds. Therefore, the time difference between microphones is almost lost between the samples themselves, as the Δt for a microphone array with spacings between 2.8 and 3.3 inches (0.083 - 0.071 meters) is on the order of 10^{-4} seconds. To better capture the frequency data over time, better hardware with a faster sampling rate is needed to properly conduct an STFT that would make this method more reliable and robust. Instead of using a Teensy and microphones with I2S, a better platform could be chosen in the future that had a more reliable serial connection. The INMP44 microphones soldered to the Teensy consistently disconnected from the Raspberry Pi, causing serial to fail during many experimental trials. Connections were inconsistent and required constant restarting, and reading from two Teensys at once (only 1 I2S port per Teensy, one port needed per microphone pair) caused latency that throttled the sampling rate. Additionally, each Teensy consumes about 0.1A of power during normal operation. The serial connection to two Teensys from the Raspberry Pi's UART USB ports meant that the robot was continuously power hungry. 2 AA batteries that were provided for the AlphaBot lasted for roughly 1 hour when attempting to stream data using Teensys. Performance of

the AlphaBot degraded with low battery, causing repeatability issues during data collection. In the future, it's recommended to only run experiments with fully charged batteries and employ another data collection method (perhaps with better microphones) that can stream I2S or I2C data with less power consumption that overwhelms the Pi.

Connection between the AlphaBot's onboard Raspberry Pi and a personal laptop to command the robot was also difficult to establish and maintain. The AlphaBots primary method of wireless connection to personal computers is SSH over a local network. At the Claremont Colleges, the network provider doesn't allow for a static IP to become established. Moving from room-to-room within Harvey Mudd caused the robot to continuously disconnect as the IP was reassigned, making the robot unable to communicate. To solve this problem, students in 2025's E205 class recommended using the Raspberry Pi connect software that allows connection over the browser instead of relying on an unstable SSH and non-static IP. This solved the connection problem and is recommended for teams to use in the future when working with the AlphaBot platform.

VI. PERFORMANCE AND APPLICATIONS

The best performance of sound localization in Kalman filtering was seen in the simulation of TDOA data and robot motion in Python (Section IV, A, Figures 12-16). As discussed in Section IV, A, this idealized simulation used low TDOA noise and a small R , which was unrealistic for the implementation that was done on AlphaBot hardware. This simulation shows that the approach to localization using TDOA and Kalman filtering is viable, but relies on highly accurate, stable, and fast-sampling hardware that was out of the scope of this project.

The primary insight of this project is the effect of the R matrix and measurement noise on the ability of Kalman filtering to produce viable state estimates. The primary differences between reliable and unreliable Kalman filter outputs in simulation lay in the use of a realistic R and TDOA noise value. These differences stemmed from the sample rate issues discussed in Section IV, B and additional issues with resolution in the INMP44 microphones.

The methods used in this project have proven to only be reliable with more expensive, fine resolution, and high-sample rate hardware, which alludes to why this method isn't widely used in establishing localization in consumer robotics. The method is highly reliable on these parameters of R and sensor measurement noise which is perhaps why sound localization isn't widely used in above-ground navigation in loud, urban environments. This method seems to be used mostly in underwater applications with low-risk (hobby robotics, unmanned AUVs, disposable surveillance robotics) or in low-resolution animal tracking.

The work outlined in this report carefully engages with the idea of sound localization and aims to act only as a learning tool. This approach with coarse data collection and off-the-shelf hardware acts to mimic the process used by similar, more accurate approaches. Since this project was not successful or innovative in its application on hardware, it is believed that

this project doesn't contribute to the active progression of sound tracking and localization problems that could be used in harmful ways.

VII. REFLECTION

The most enriching part of this project was working through the full sensing and estimation pipeline, particularly sensor communication and data streaming, data processing, and reasoning through the Kalman filter. Unlike more structured assignments, we were not given fixed parameters or a fully defined system, and instead had to design and debug each component ourselves, from acquiring audio data to converting it into position estimates. This made the project both challenging and rewarding, as it required a deeper understanding of how each layer of the system interacts. At the same time, we encountered several unanticipated challenges, including inconsistent robot behavior, battery-related issues, and failures to properly initialize the connection client. On the sensor side, we faced difficulties with Teensy streaming reliability and correctly indexing UART ports, which further complicated data collection. Given additional time, we would improve the system by implementing the Kalman filter directly onboard the robot to enable real-time correction and visualization of its trajectory. We would also calibrate systematic motion biases, such as the robot consistently drifting to the left. If approaching the problem again, we would like to redesign the sensing setup to include different emitters and a fully time-synchronized system between the Raspberry Pi and the beacons. Knowing the exact emission time and corresponding time of arrival at each microphone would allow us to validate whether measured delays are physically consistent with the beacon-to-robot distances. In the current approach, we rely only on relative time differences between microphones, which limits verification the correctness of the measured signals.

A. Ethical Considerations of Sound Localization

When working within robotics it's important to consider the ethical implications of work. In using Kalman filtering for state estimation, it is inherent that a sound, frequency, or connection signal is used as a data source or serves as an object being tracked [3]. This could pose consequences in the realm of tracking a human using sound or other emitted frequencies, such as cellphones. Tracking other people based on sounds they make or frequencies their devices emit poses issues in personal safety, data privacy, and military operations [3]. These techniques have historically been used in violent or unsafe situations since these methods may be accurate and undetectable [4]. Additionally, acoustic localization is heavily used in navigating and tracking devices underwater. Parties interested in tracking the movement of aquatic objects such as submarines, boats, or large ships can utilize the ability of water to transmit sound and the objects reflection, emittance, or absorption of acoustic waves to track their position within a global frame. The Navy uses these techniques to monitor and track the position of its own vehicles and other devices, aiding in the surveillance of other countries or parties [4]. Military

surveillance is a large use case for underwater acoustic or sound tracking, specifically listening for low-frequencies that travel long distances [4]. SOSUS is a network of hydrophones in the Pacific Ocean owned by the US Navy that picks up these low frequencies as part of their underwater surveillance techniques [1]. This technique is also used in underwater robotics, as pings from a host research vessel or surveying ship can acoustically communicate with a deployed underwater device via systems called USBLs, which stands for UltraShort Baseline [9]. Underwater robotics may also contain acoustic pings that could localize a lost device in a reverse method to the ones outlined in this report (using three or more microphones to localize one sound emitter). This reverse localization approach is also commonly used in the area of underwater tracking of animals for insight into their lifetime migratory patterns and health [8]. Sharks, turtles, sea lions, and other engaged aquatic life are often tagged with acoustic trackers that allow researchers to "listen" for their position in different areas of the ocean [8]. Acoustic microphones record pings from devices of a certain frequency (such as an animal tag frequency, rather than a ship motor's noise) and using nearby emitters and a similar TDOA/TOF approach can localize the location of the ping based on the known locations of the underwater microphones [9].

VIII. APPENDIX

To see our code, access our GitHub repository here. And to see the simulation, access it here.

REFERENCES

- [1] Commander, Submarine Group Ten. Origins of sosus. <https://www.csp.navy.mil/cus/About-IUSS/Origins-of-SOSUS/>, n.d. Accessed: 2026-05-06.
- [2] Mikael Drakopoulos. Hyperbolas of the tdoa method with three base station receivers. https://www.researchgate.net/figure/Hyperbolas-of-the-TDOA-method-with-three-base-station-receivers_fig2_309846689, 2016. Accessed: 2026-05-06.
- [3] Electronic Privacy Information Center. Epic v. fbi – stingray cell site simulator. <https://epic.org/documents/epic-v-fbi-stingray-cell-site-simulator/>, n.d. Accessed: 2026-05-06.
- [4] Federation of American Scientists. Sosus: Sound surveillance system. <https://irp.fas.org/program/collect/sosus.htm>, n.d. Accessed: 2026-05-06.
- [5] GrabCAD. Alphabot2-ar. <https://grabcad.com/library/alphabot2-ar-1>, n.d. Accessed: 2026-05-06.
- [6] GrabCAD. Inmp441 i2s microphone module. <https://grabcad.com/library/inmp441-i2s-microphone-module-1>, n.d. Accessed: 2026-05-06.
- [7] MathWorks. Object tracking using time difference of arrival (tdoa). <https://www.mathworks.com/help/fusion/ug/object-tracking-using-time-difference-of-arrival.html>, n.d. Accessed: 2026-05-06.
- [8] NOAA Fisheries, Endangered Species Conservation. Passive acoustic research in the northeast. <https://www.fisheries.noaa.gov/new-england-mid-atlantic/endangered-species-conservation/passive-acoustic-research-northeast>, n.d. Accessed: 2026-05-06.
- [9] Sonardyne International Ltd. What is a usbl and how does it work? <https://www.sonardyne.com/products-knowledge-base/what-is-a-usbl-and-how-does-it-work/>, n.d. Accessed: 2026-05-06.